INTELIGÊNCIA ARTIFICIAL 2026.1
Prof. Felipe B. da Silva, Ph.D.

Lab 1 – Implementação de KNN (Aprendizado de Máquina Supervisionado)

Instruções:

1. Leia todo o material.
2. Responda às questões organizadamente e sequencialmente.
3. Entregue um relatório único e completo contemplando todas as questões requeridas.

Objetivos:

- Aprender a implementação em Python de um algoritmo (KNN) de aprendizado de máquina supervisionado.
- Entender a separação de dados em treino, teste e validação.
- Interpretar matrizes de confusão.
- Compreender métricas de avaliação da performance do algoritmo.
- Interpretar dados e resultados.

Lab 1 – Implementação de KNN (Aprendizado Supervisionado)

1. Associe cada bloco de código Python à etapa mais apropriada a fim de implementar o KNN sobre o conjunto de dados de Íris.

 # Etapa 0: Importando pacotes -> B2

B2.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

 # Etapa 1: Carregar o conjunto de dados Íris -> B5

B5.

```
data = load_iris()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target

# Exibir as cinco primeiras linhas do conjunto de dados
print("Visualização inicial do dataset:")
print(df.head())
```

 # Etapa 2: Análise exploratória dos dados ->

B1.

```
sns.pairplot(df, hue='target', palette='Dark2')
plt.show()
```

 # Etapa 3: Separação em Conjunto de Treino e Teste -> B7

B7.

```
X = df.iloc[:, :-1]  # Features
y = df['target']     # Target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

 # Etapa 4: Padronização dos Dados -> B6

B6.

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Etapa 5: Treinamento e Avaliação do KNN ->
B4.
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)

# Avaliação
print("Acurácia do modelo KNN:", accuracy_score(y_test, y_pred))
print("Matriz de Confusão:")
print(confusion_matrix(y_test, y_pred))
print("Relatório de Classificação:")
print(classification_report(y_test, y_pred))

# Etapa 6: Testando Diferentes Valores de k ->
B3.
accuracies = []
k_values = range(1, 21)
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    accuracies.append(accuracy_score(y_test, y_pred))

# Plotando a acurácia para diferentes valores de k
plt.plot(k_values, accuracies, marker='o', linestyle='dashed', color='b')
plt.xlabel('Número de Vizinhos (k)')
plt.ylabel('Acurácia')
plt.title('Impacto de k no Modelo KNN')
plt.show()
```

2. Implemente o Código completo em Python.

Código em Python implementado na ordem fornecida no ultimo exercício.

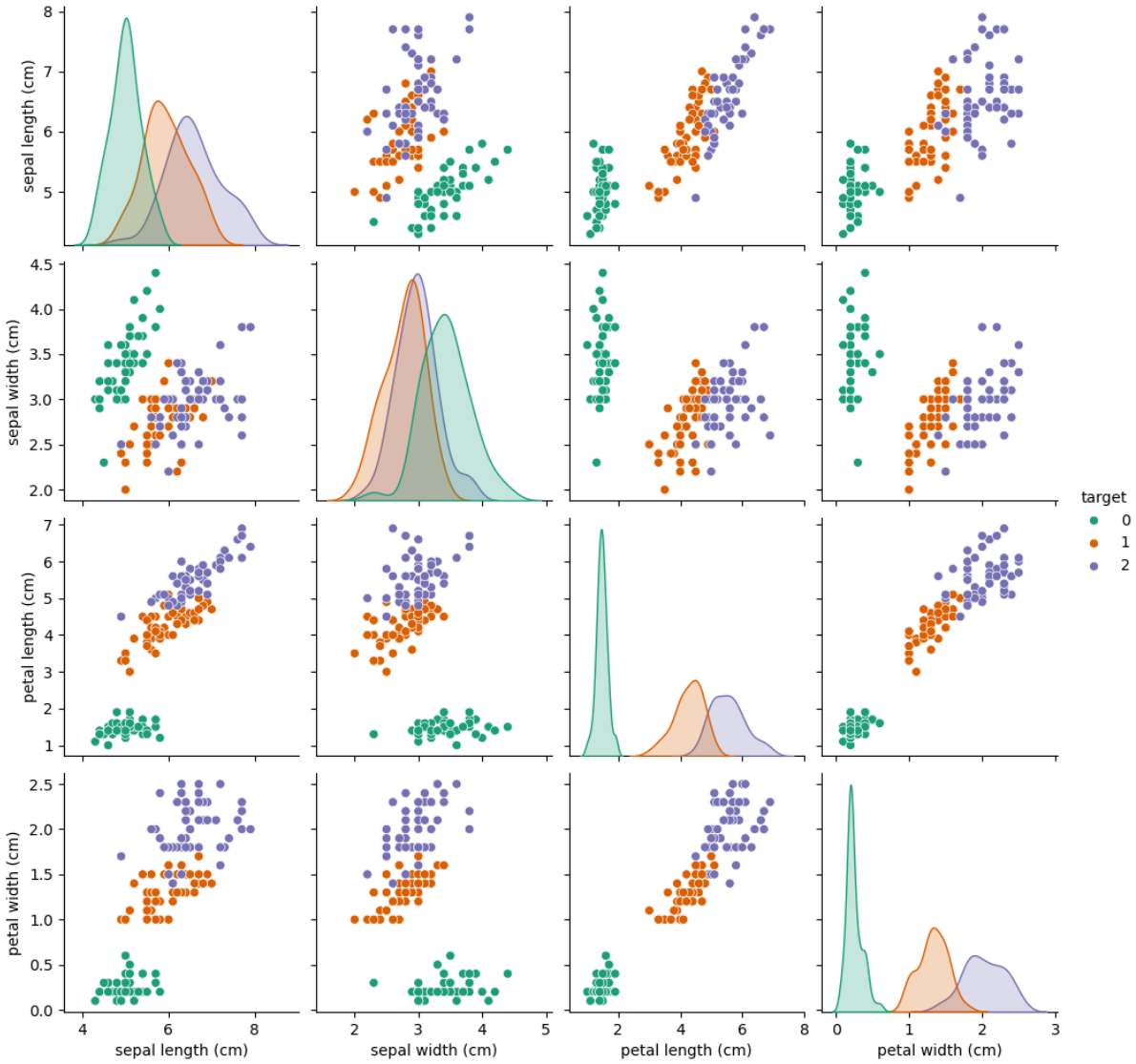
3. Interprete a matriz de confusão.

A matriz de confusão permite visualizar os acertos e erros do modelo em um formato de tabela, em que na diagonal (da esquerda em cima até direita em baixo) há os valores que foram corretamente classificados, e os valores em outros pontos da matriz mostram classificação errada do modelo relacionando a classificação certas.
O código da maneira que foi apresentado possuía um volume de teste de 0.2, totalizando 30 testes, e a matriz de confusão possuía esses 30 testes distribuídos na diagonal, ou seja, não houveram erros nessas condições.

4. Defina o que são precisão, acurácia, F1- score, e suporte.

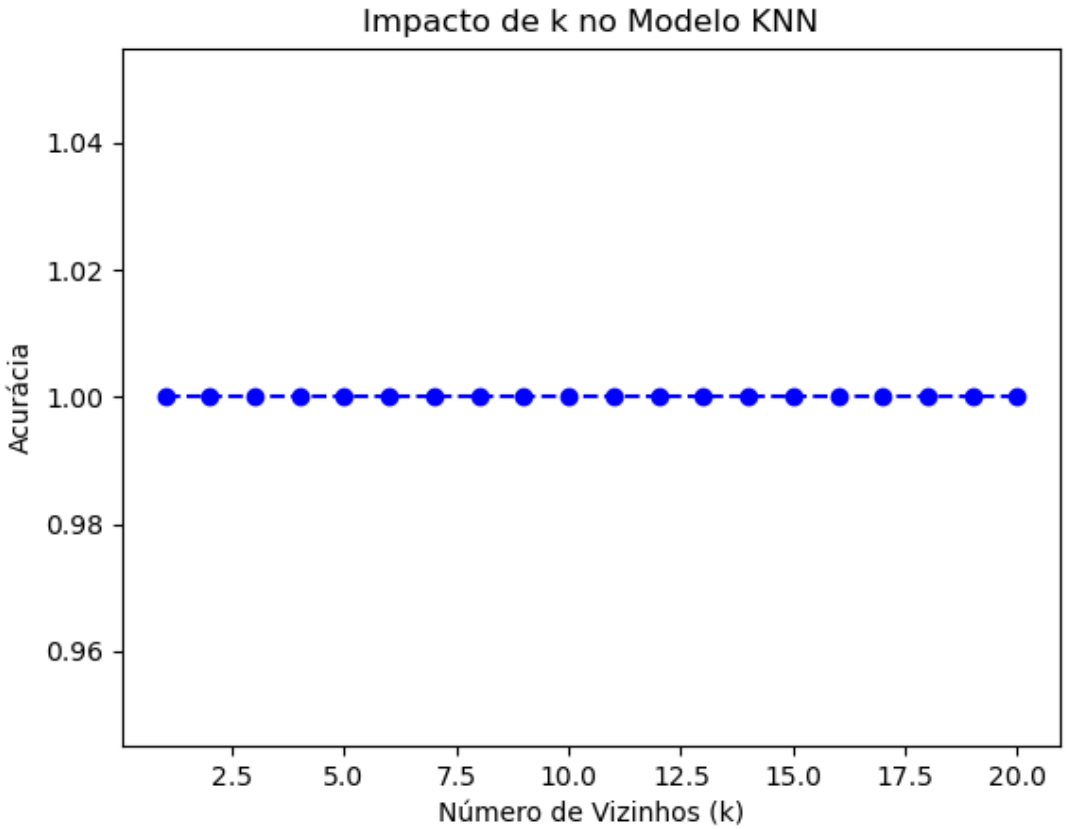
A precisão é uma forma de medir de todas as previsões do modelo, quantas estavam certas.
Já a acurácia avalia de uma forma inversa, de todos os casos corretos, quantos que o modelo acertou.
E o F1-score é uma forma de verificar se o modelo realmente está realizando as classificações corretamente de forma que relaciona tanto a precisão quanto a acurácia (recall) a fim de determinar os acertos. Apenas uma acurácia boa, ou apenas a precisão boa, não determinam que o modelo classifica de forma correta, e o F1-score consegue relaciona-los.
Já o suporte é o numero de classificações corretas do modelo para cada classe, sendo assim a soma dos três suportes é equivalente ao tamanho do teste.

5. Interprete o gráfico gerado.



O primeiro gráfico gerado se refere á base de dados em si, relaciona os parâmetros de classificação das espécies de flores, e as identifica conforme a legenda,

estabelecendo uma relação básica entre esses parâmetros e as características reais de cada classe (ex: 0 – possui pétalas pequenas e finas, e sépalas pequenas e largas), com os gráficos na diagonal representando uma distribuição Gaussiana dos dados.



Já o segundo gráfico apresenta uma relação entre a acurácia e o numero de vizinhos, permite avaliar qual o numero de vizinhos ideal para realizar o treinamento do modelo, nesse caso inicial, os parâmetros do treinamento permitiram uma situação que qualquer valor de k até 21 possui uma acurácia perfeita.
Ao alterar parâmetros como tamanho de teste ou randomização dos dados, o ambiente apresenta outras características em que determinado K possui acurácia maior, o tornando em um numero ideal para treinamento.

6. **Faça um pequeno texto sumarizando toda atividade apresentando as principais definições, explicando a implementação do algoritmo. Por fim, faça uma análise dos resultados.**

A atividade é uma aplicação real de um modelo KNN utilizando bibliotecas do python, ela importa uma base de dados referente á parâmetros de classificação da flor Iris (tamanho e largura das pétalas e sépalas), utiliza o numpy e pandas para a visualização dos gráficos gerados, o código importa as dependências (bibliotecas matemáticas, gráficas e knn; e os dados que seram convertidos em datasets, para análise e transformação em gráficos) e é gerada a visualização desses parâmetros e classificações, o código em seguida estabelece os parâmetros do treinamento e normaliza os dados para uma classificação correta por parte dos modelos.

Em seguida, realiza o treinamento com os métodos da biblioteca de KNN, realiza testes, dos valores de K, para gerar a visualização e análise do usuário de qual o melhor parâmetro a ser utilizado.
A atividade permite o entendimento dos conceitos de matriz de confusão, precisão, acurácia(ou recall), F1-score e suporte, como ferramentas de avaliação e análise de desempenho do modelo treinado e como a mudança de parâmetros implica no desempeno do modelo (ex, alto volume de testes, implica em um menor volume de treinamento, gerando um modelo que erra mais as classificações).

Referências

IBM. **O que é uma matriz de confusão?** [2024]. Disponível em: https://www.ibm.com/br-pt/think/topics/confusion-matrix. Acesso em: 16 abr. 2026.